

Lab1: Write a C# console program that defines a class Student with properties Name, Roll, and Marks using auto-implemented properties, and demonstrate object creation along with a constructor that initializes all three fields.

Source code:

```
using System;

class Student
{
    public string Name { get; set; }
    public int Roll { get; set; }
    public double Marks { get; set; }

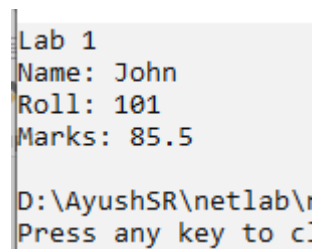
    public Student(string name, int roll, double marks)
    {
        Name = name;
        Roll = roll;
        Marks = marks;
    }
}

class Program
{
    static void Main()
    {
        Console.WriteLine("Lab 1");

        Student s = new Student("John", 101, 85.5);

        Console.WriteLine($"Name: {s.Name}");
        Console.WriteLine($"Roll: {s.Roll}");
        Console.WriteLine($"Marks: {s.Marks}");
    }
}
```

Output:



```
Lab 1
Name: John
Roll: 101
Marks: 85.5

D:\AyushSR\netlab\
Press any key to c:
```

Lab2: Create a base class Shape with a virtual method Area() and derive classes Circle and Rectangle that override Area(). Demonstrate runtime polymorphism by calling Area() through a base class reference.

Source code:

```
using System;

class Shape
{
    public virtual double Area() => 0;
}

class Circle : Shape
{
    public double Radius;

    public Circle(double radius) => Radius = radius;

    public override double Area() => Math.PI * Radius * Radius;
}

class Rectangle : Shape
{
    public double Length, Width;

    public Rectangle(double length, double width)
    {
        Length = length;
        Width = width;
    }

    public override double Area() => Length * Width;
}

class Program
{
    static void Main()
    {
        Console.WriteLine("Lab 2");

        Shape s = new Circle(5);
        Console.WriteLine("Circle Area: " + s.Area());

        s = new Rectangle(4, 6);
        Console.WriteLine("Rectangle Area: " + s.Area());
    }
}
```

Output:

```
Lab 2
Circle Area: 78.53981633974483
Rectangle Area: 24
D:\AyushSR\netlab\netlab\bin\De
```

Lab3: Write a program using an indexer in a class Week that allows accessing day names using an integer index (e.g., week[0] returns "Sunday").

Source code:

```
using System;

class Week
{
    private string[] days =
    { "Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"
    };

    public string this[int index]
    {
        get { return days[index]; }
    }
}

class Program
{
    static void Main()
    {
        Console.WriteLine("Lab 3");

        Week week = new Week();

        Console.WriteLine(week[0]);
        Console.WriteLine(week[1]);
        Console.WriteLine(week[2]);
    }
}
```

Output:

```
Lab 3
Sunday
Monday
Tuesday
D:\AyushSR\net.
Press any key t
```

Lab4: Implement a generic class Stack<T> with Push, Pop, and Peek methods, and demonstrate its use with both int and string data types.

Source code:

```
using System;
using System.Collections.Generic;

class Stack<T>
{
    private List<T> items = new List<T>();

    public void Push(T item) => items.Add(item);

    public T Pop()
    {
        T item = items[items.Count - 1];
        items.RemoveAt(items.Count - 1);
        return item;
    }

    public T Peek() => items[items.Count - 1];
}

class Program
{
    static void Main()
    {
        Console.WriteLine("Lab 4");

        Stack<int> numbers = new Stack<int>();
        numbers.Push(10);
        numbers.Push(20);
        Console.WriteLine("Int Peek: " + numbers.Peek());
        Console.WriteLine("Int Pop: " + numbers.Pop());

        Stack<string> words = new Stack<string>();
        words.Push("Hello");
        words.Push("World");
        Console.WriteLine("String Peek: " + words.Peek());
        Console.WriteLine("String Pop: " + words.Pop());
    }
}
```

Output:

```
Lab 4
Int Peek: 20
Int Pop: 20
String Peek: World
String Pop: World

D:\AyushSR\netlab\net1
```

Lab5: Write a C# program demonstrating the use of an abstract class Employee with an abstract method CalculateSalary(), implemented differently by two derived classes Manager and Clerk.

Source code:

```
using System;

abstract class Employee
{
    public abstract double CalculateSalary();
}

class Manager : Employee
{
    public override double CalculateSalary() => 50000;
}

class Clerk : Employee
{
    public override double CalculateSalary() => 25000;
}

class Program
{
    static void Main()
    {
        Console.WriteLine("Lab 5");

        Employee e;

        e = new Manager();
        Console.WriteLine("Manager Salary: " + e.CalculateSalary());

        e = new Clerk();
        Console.WriteLine("Clerk Salary: " + e.CalculateSalary());
    }
}
```

Output:

```
Lab 5
Manager Salary: 50000
Clerk Salary: 25000

D:\AyushSR\netlab\net1
Press any key to close
```

Lab6: Write a program that defines an interface IPayable with a method Pay(), implement it in two classes Invoice and Salary, and call the method polymorphically using a List<IPayable>.

Source code:

```
using System;
using System.Collections.Generic;

interface IPayable
{
    void Pay();
}

class Invoice : IPayable
{
    public void Pay() => Console.WriteLine("Invoice Paid");
}

class Salary : IPayable
{
    public void Pay() => Console.WriteLine("Salary Paid");
}

class Program
{
    static void Main()
    {
        Console.WriteLine("Lab 6");

        List<IPayable> payments = new List<IPayable>
        {
            new Invoice(),
            new Salary()
        };

        foreach (IPayable p in payments)
            p.Pay();
    }
}
```

Output:

```
Lab 6
Invoice Paid
Salary Paid

D:\AyushSR\net1
Press any key t
```

Lab7: Demonstrate the use of delegates and events in C# by creating a simple event that notifies subscribers when a temperature value crosses a threshold.

Source code:

```
using System;

delegate void TemperatureHandler();

class Temperature
{
    public event TemperatureHandler TemperatureExceeded;

    public void CheckTemperature(int value)
    {
        if (value > 30)
            TemperatureExceeded?.Invoke();
    }
}

class Program
{
    static void Main()
    {
        Console.WriteLine("Lab 7");

        Temperature temp = new Temperature();

        temp.TemperatureExceeded += () =>
            Console.WriteLine("Temperature crossed the threshold!");

        temp.CheckTemperature(35);
    }
}
```

Output:

```
Lab 7
Temperature crossed the threshold!
D:\AyushSR\netlab\netlab\bin\Debug\
```

Lab8: Write a program using LINQ and lambda expressions to filter a list of integers and return only even numbers greater than 10, sorted in descending order.

Source code:

```
using System;
using System.Collections.Generic;
using System.Linq;

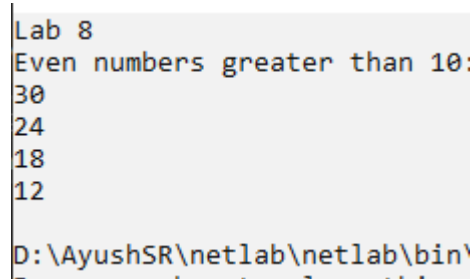
class Program
{
    static void Main()
    {
        Console.WriteLine("Lab 8");

        List<int> numbers = new List<int> { 5, 12, 18, 7, 24, 10, 30, 15 };

        var result = numbers
            .Where(n => n > 10 && n % 2 == 0)
            .OrderByDescending(n => n);

        Console.WriteLine("Even numbers greater than 10:");
        foreach (int n in result)
            Console.WriteLine(n);
    }
}
```

Output:



```
Lab 8
Even numbers greater than 10:
30
24
18
12
D:\AyushSR\netlab\netlab\bin\
```

Lab9: Write a C# program that reads data from a text file and writes processed output to another file using File I/O classes (StreamReader/StreamWriter), handling exceptions using try-catch-finally.

Source code:

```
using System;
using System.IO;
class _09_FileIOAndExceptionHandling
{
    static void Main()
    {
        string inputFile = "student.txt";
        string outputFile = "processed-student.txt";

        try
        {
            using (StreamWriter writer = new StreamWriter(inputFile))
                writer.WriteLine("My name is ayush and roll number is : 6");

            Console.WriteLine("Data written successfully.");

            string data;

            using (StreamReader reader = new StreamReader(inputFile))
                data = reader.ReadToEnd();

            Console.WriteLine("Data read from file:");
            Console.WriteLine(data);

            using (StreamWriter writer = new StreamWriter(outputFile))
            {
                writer.WriteLine("Processed Data:");
                writer.WriteLine(data.ToUpper());
            }

            Console.WriteLine("Processed data written successfully.");
        }
        catch (Exception e)
        {
            Console.WriteLine("Error: " + e.Message);
        }
        finally
        {
            Console.WriteLine("File operation completed.");
        }
    }
}
```

Output:

```
Data written successfully.
Data read from file:
My name is ayush and roll number is : 6

Processed data written successfully.
File operation completed.

D:\AyushSR\netlab\netlab\bin\Debug\net8.
Press any key to close this window
```

Lab10: Write an asynchronous C# program using async/await that simulates downloading two files concurrently and prints a message once both downloads are complete.

Source code:

```
using System;
using System.Threading.Tasks;

class Program
{
    static async Task DownloadFile(string file)
    {
        Console.WriteLine("Downloading " + file + "...");
        await Task.Delay(2000);
        Console.WriteLine(file + " downloaded.");
    }

    static async Task Main()
    {
        Console.WriteLine("Lab 10");

        Task file1 = DownloadFile("File1");
        Task file2 = DownloadFile("File2");

        await Task.WhenAll(file1, file2);

        Console.WriteLine("Both downloads are complete.");
    }
}
```

Output:

```
Lab 10
Downloading File1...
Downloading File2...
File1 downloaded.
File2 downloaded.
Both downloads are complete.
D:\AyushSR\netlab\netlab\bin\
```